

Agent Harnesses

How teams keep Copilot useful, bounded, and reviewable.

Rules

Moves

Tools

Checks



A good prompt helps once. A harness helps every session.

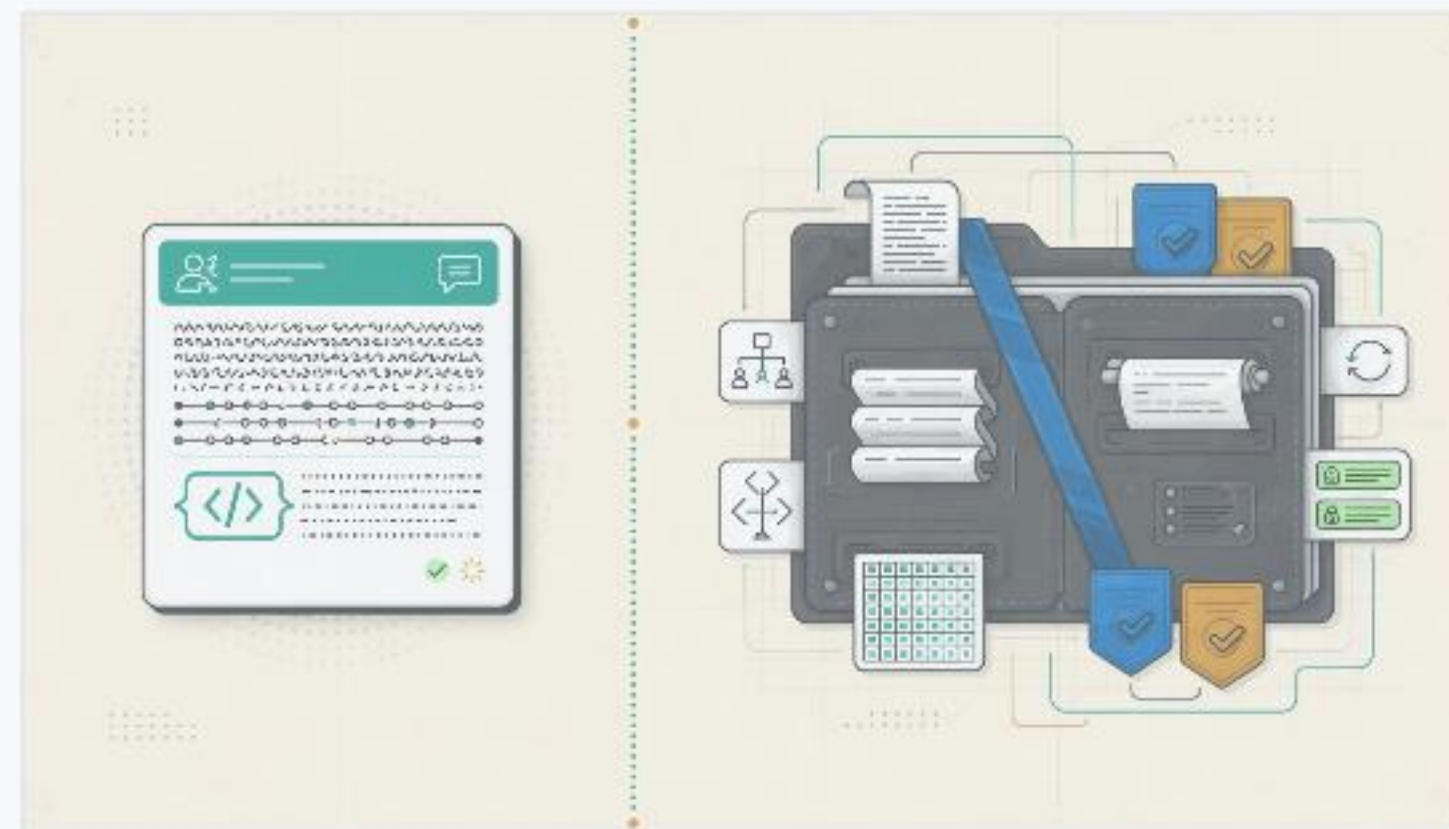
The repo should carry the rules, checks, and habits the agent should not rediscover.

One session

Prompt, context, and steering.

Every session

Instructions, skills, scripts, checks, and review habits.



The goal is not more trust. The goal is more inspectable work.

Use four pieces: rules, moves, tools, checks.

Rules

Standing instructions and boundaries.

Moves

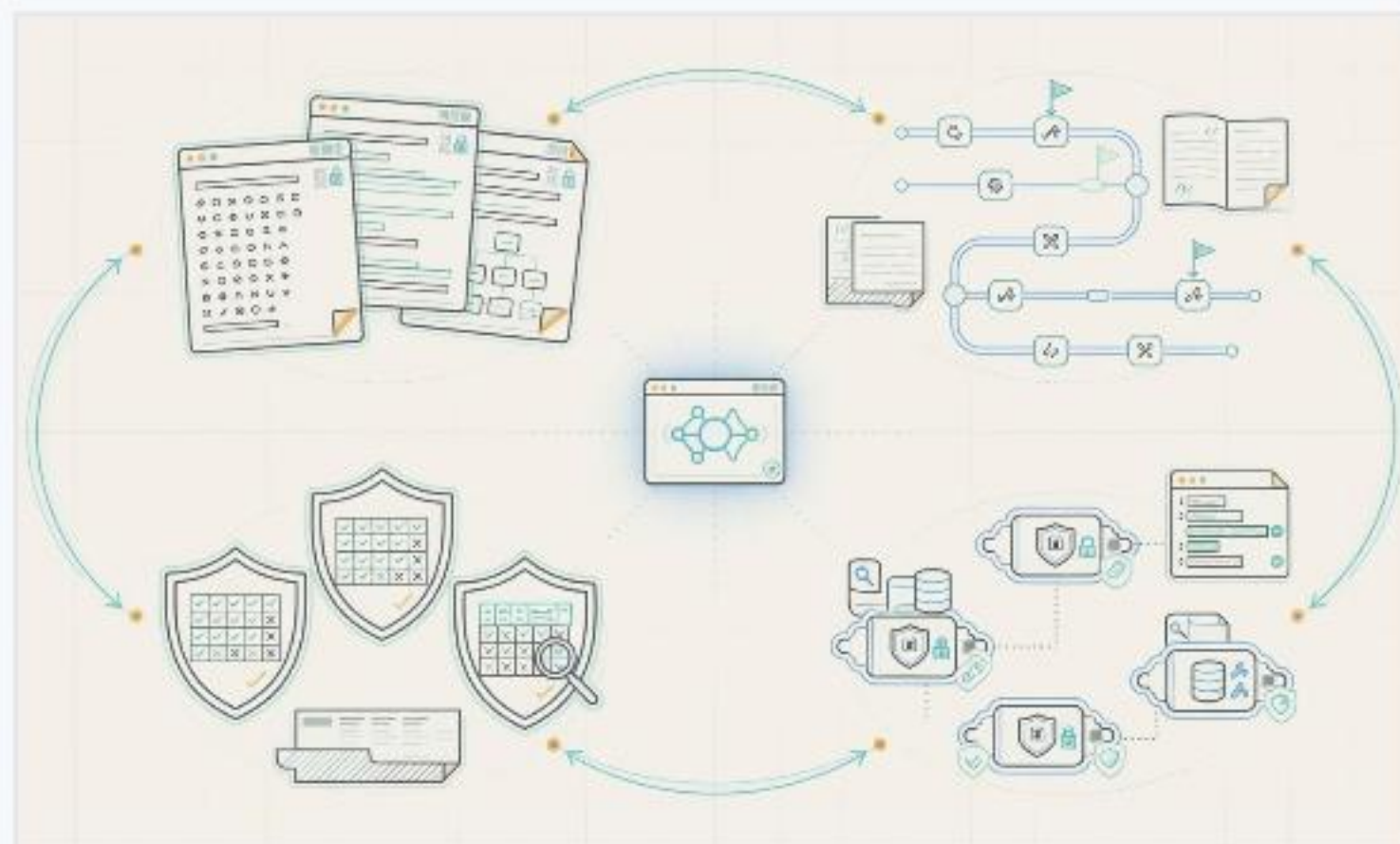
Skills, playbooks, and team procedures.

Tools

Scripts, commands, and controlled capabilities.

Checks

Tests, lint reports, diagnostics, and review.



A harness is the system around Copilot, not one magic prompt.

Repo instructions are standing orders.

`.github/copilot-instructions.md`

Write the rules the agent should not guess every time.

Commands

Install, build, lint, test, and local checks.

Boundaries

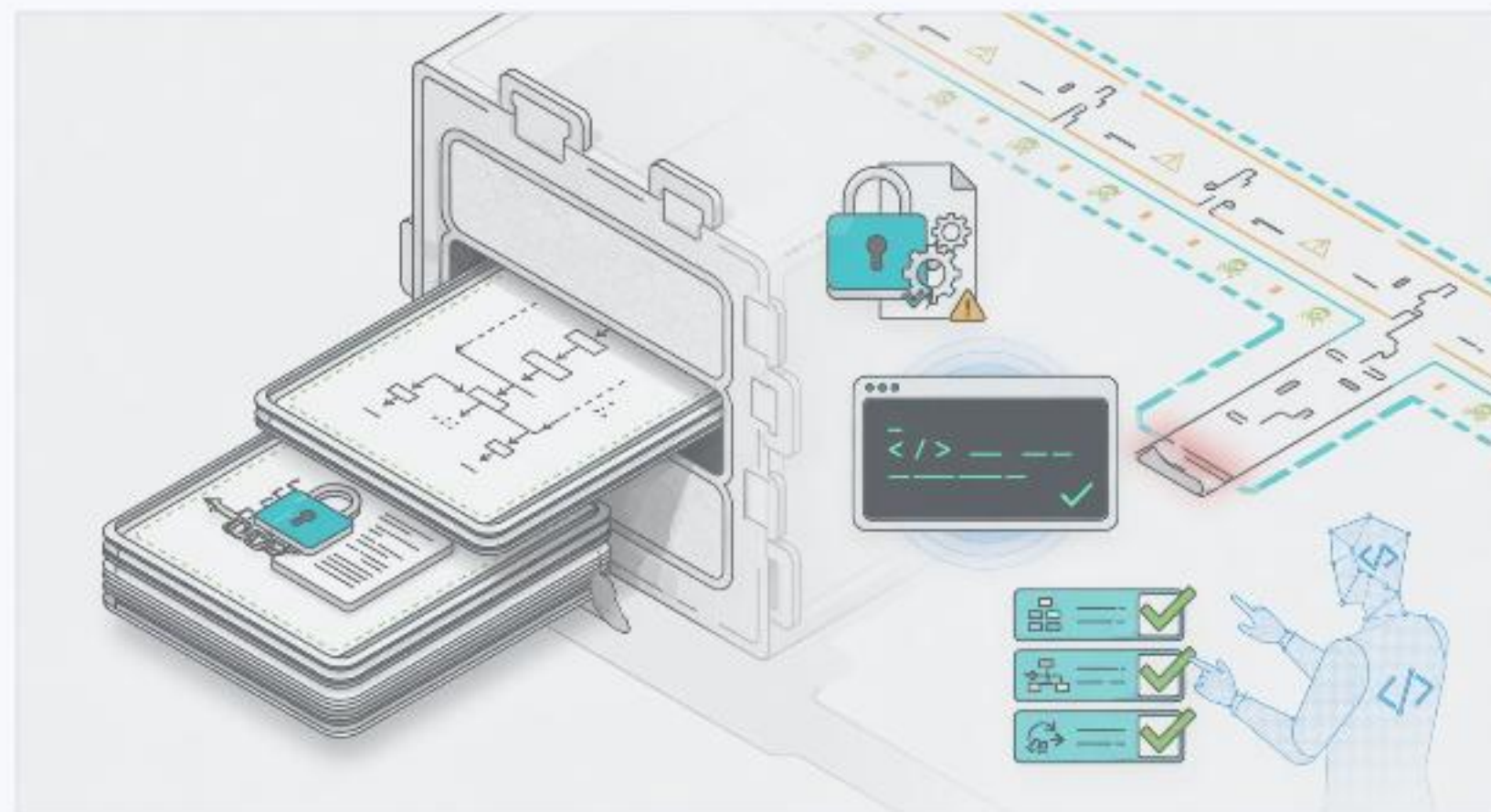
Risky paths, APIs, secrets, migrations, and deploys.

Conventions

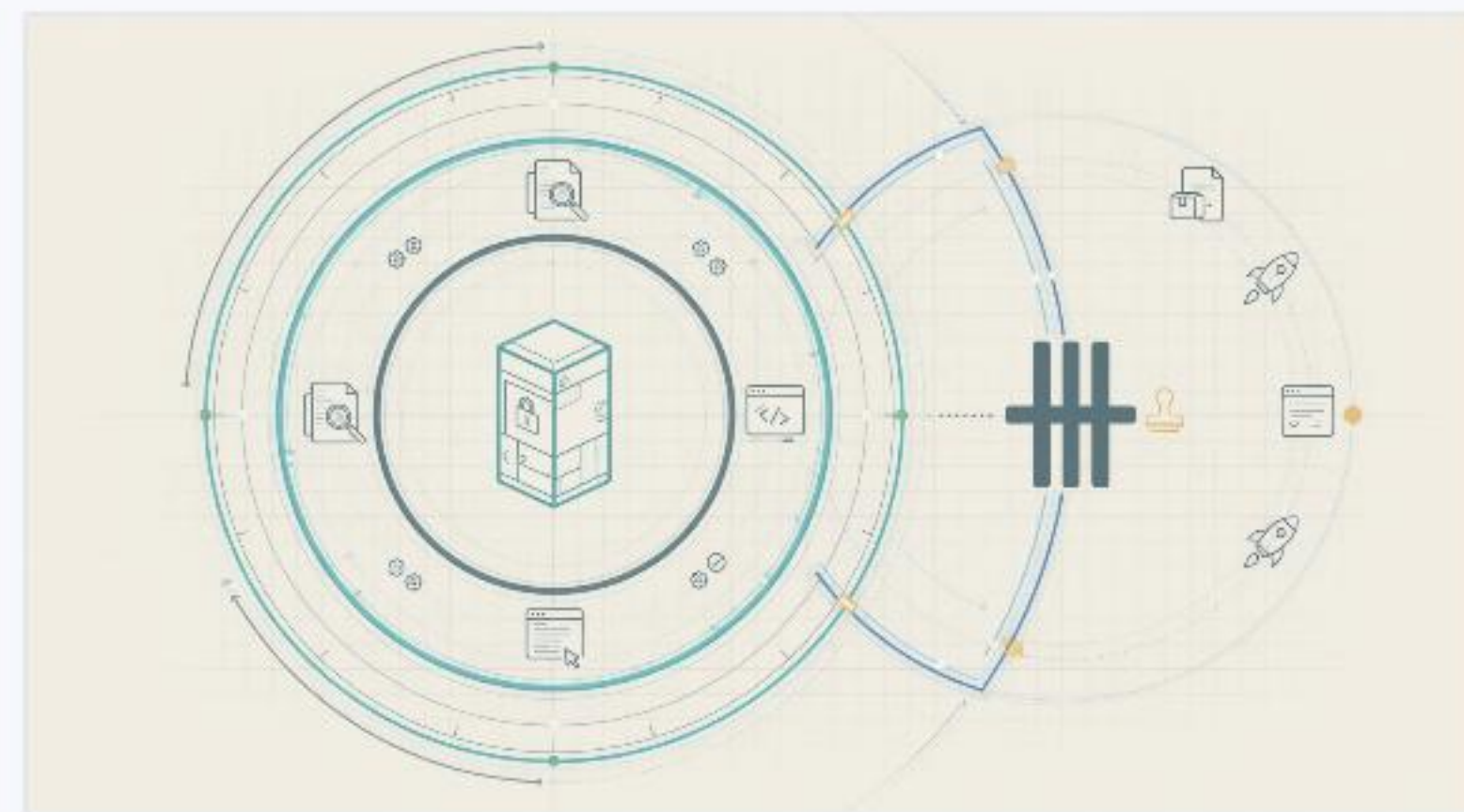
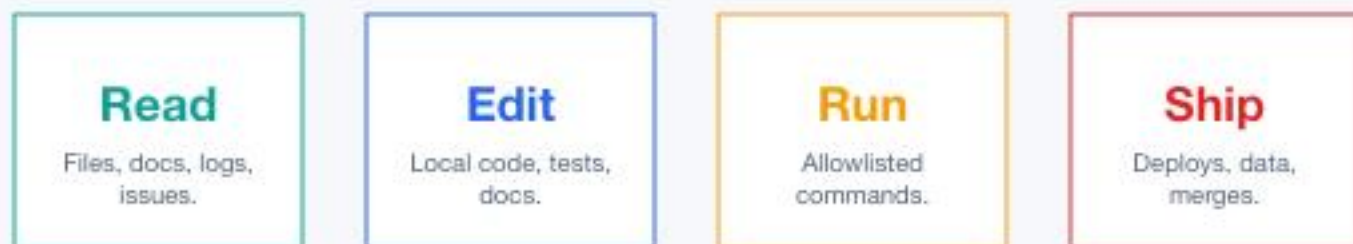
Patterns, helpers, naming, errors, and logging.

Done

Required checks before Copilot says complete.



Different actions deserve different permission.



Allowed

Read code, inspect tests, update focused local files.

Ask first

Install packages, delete files, migrations, or public API changes.

Never alone

Secrets, production data, releases, or protected branches.

Boundaries make Copilot safer and make the search space smaller.

Skills teach repeatable moves.

Instructions define the rules. Skills define the moves.

Before coding

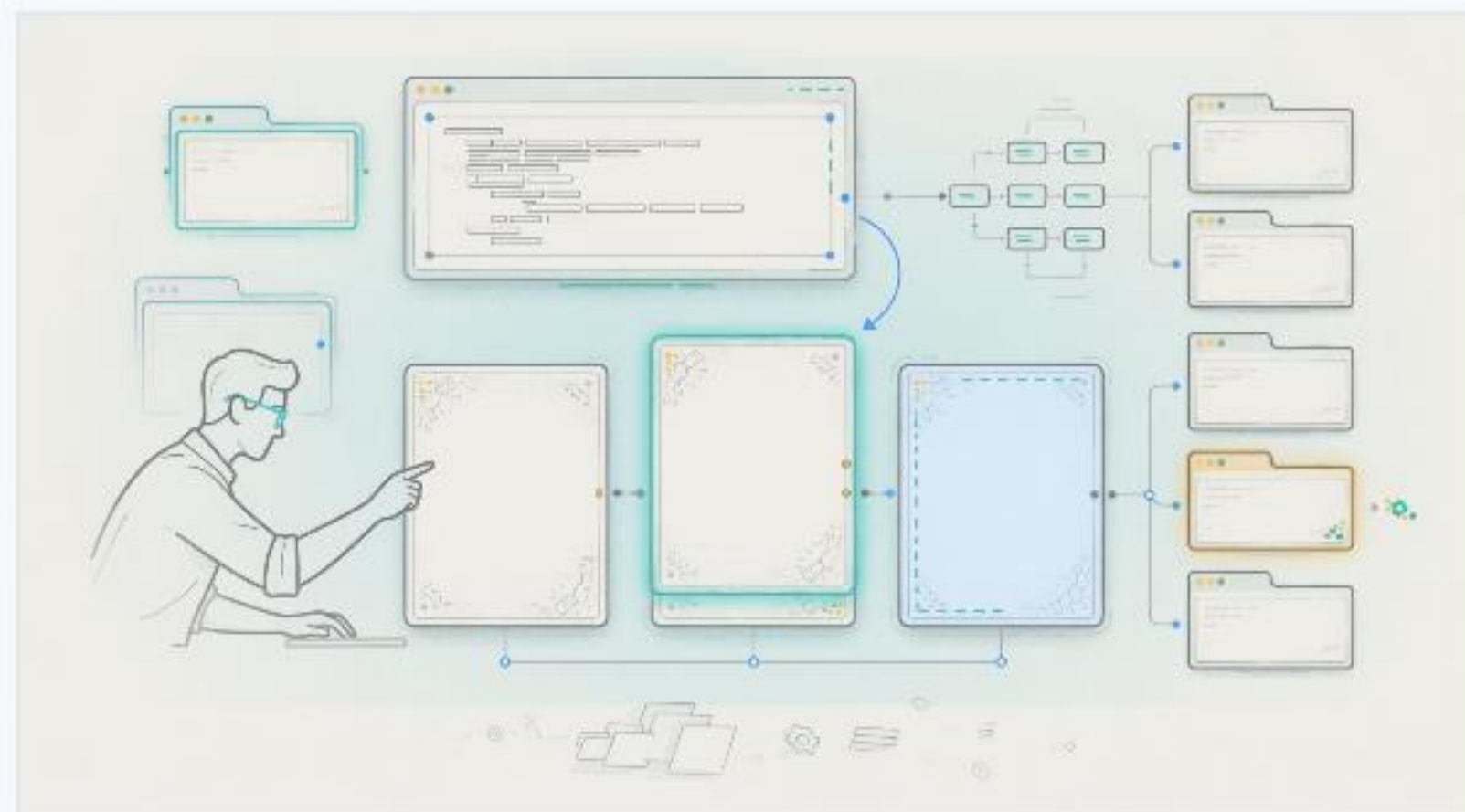
Stress-test the plan, scope, and missing evidence.

After coding

Diagnose the generated feature and hunt for bugs.

During review

Check auth, UI, migrations, or release notes with the team recipe.



grill-me

diagnose

review-auth

ui-smoke

A harness captures how your team works.

Bug triage

Start with evidence, owner, repro, and one likely surface.

UI testing

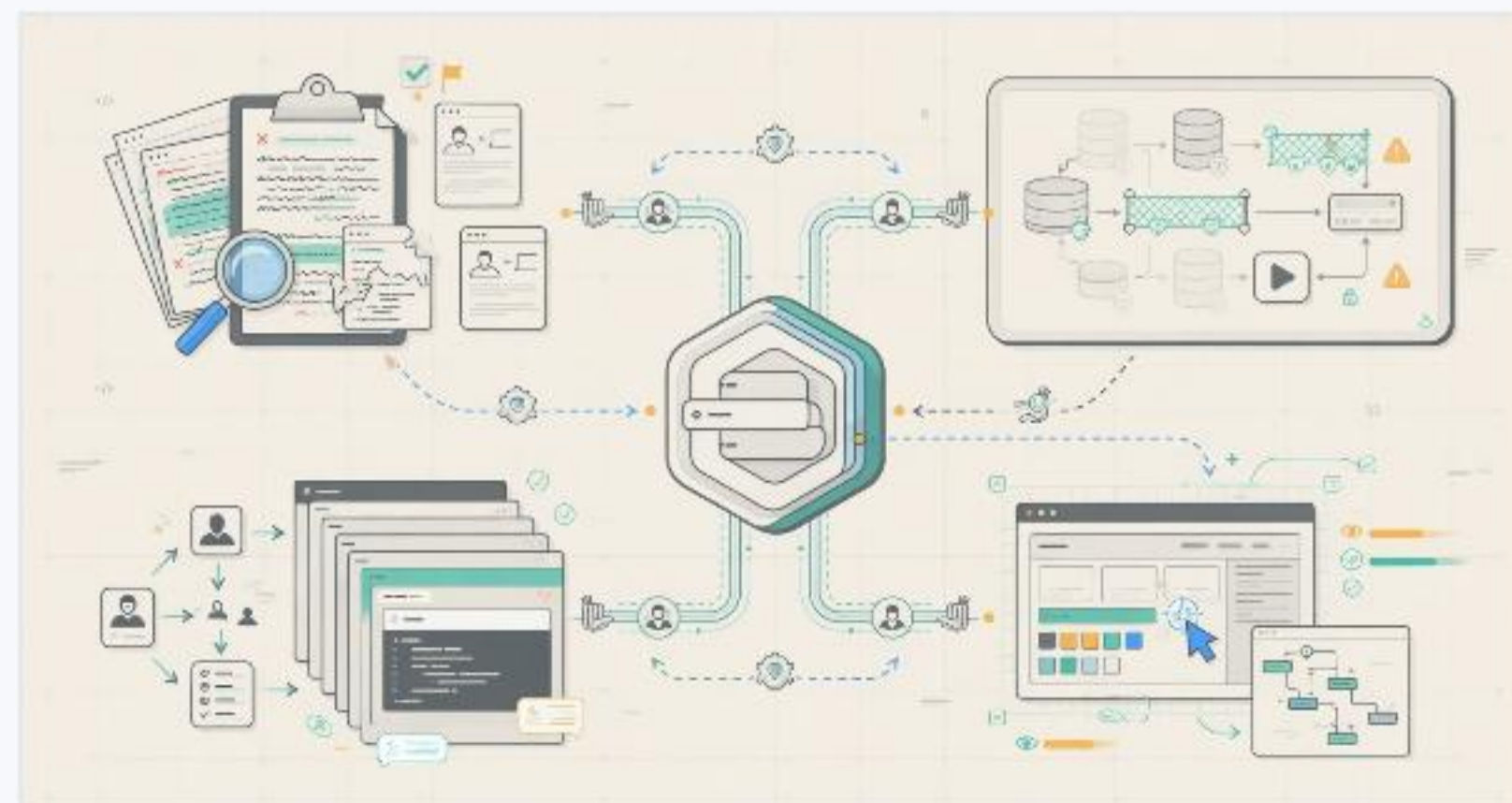
Run the app, inspect the screen, capture the visual result.

Migration safety

Dry-run first. Human approval before data-changing commands.

PR review

Summarize evidence, behavior changes, checks, and risk.



If the team repeats a move, make it part of the harness.

Give Copilot safe paths through verification.

A named script is better than asking the agent to invent the right command.

npm run test:changed

Run only tests related to changed files.

npm run agent:verify

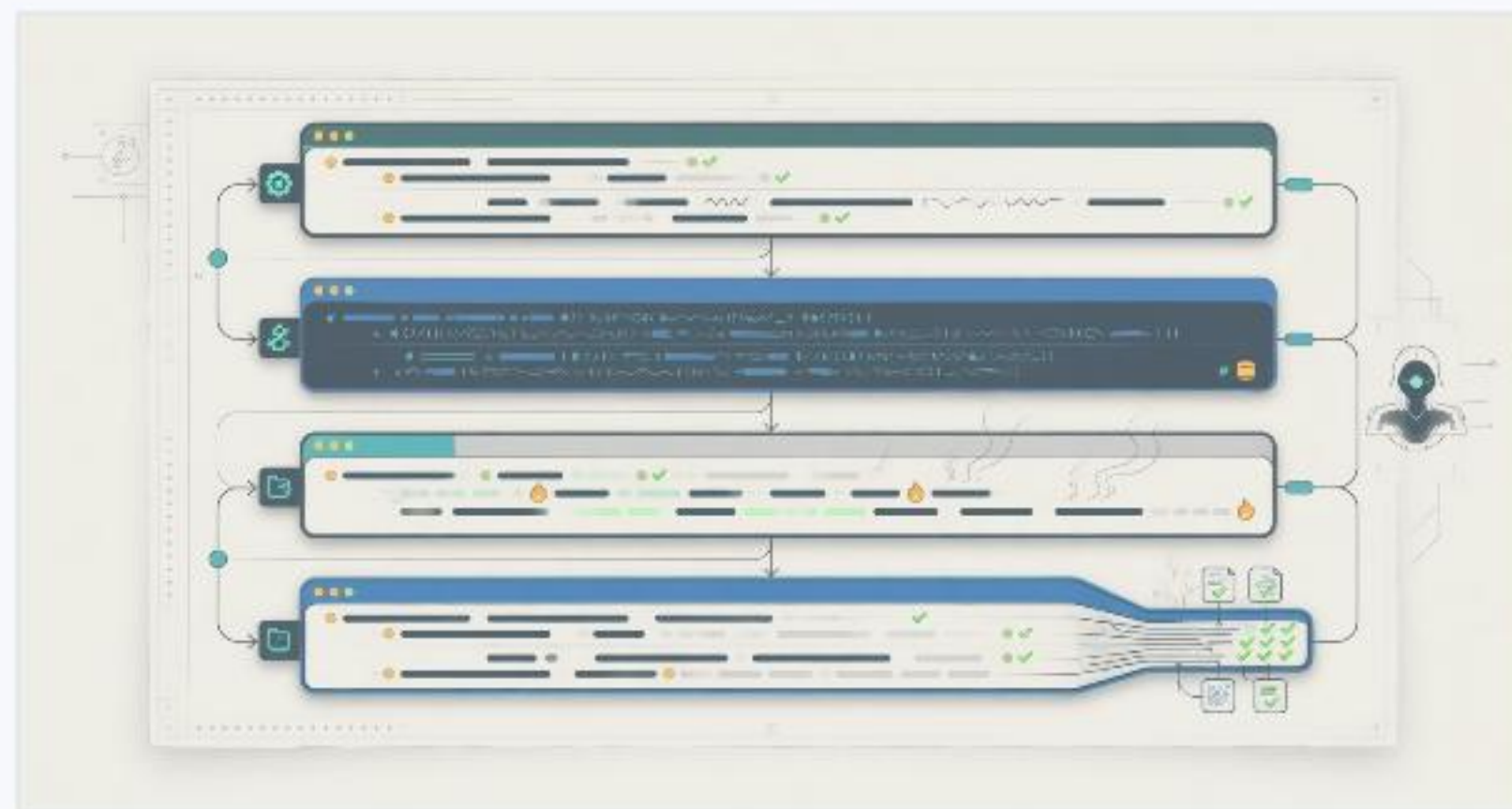
Bundle typecheck, lint report, tests, and smoke checks.

npm run smoke:local

Open the app path and confirm the main behavior still works.

npm run check:pr

Collect review evidence before merging.



Warnings can be a useful middle ground.

Not every repo has hard lint gates.
Agents still need feedback.

Hard gate

Blocks merge when the repo already enforces the rule.

Soft report

Returns warnings so Copilot can fix obvious issues without stopping.

unused code

unsafe any

accessibility hints

slow tests

wide diff



A soft check is not permission to ignore quality. It is a way to surface quality earlier.

Inspect before review.

Human review starts from evidence, not from a confident summary.



Generate

Copilot edits code inside the task boundary.



Run checks

Changed tests, soft lint, typecheck, and smoke commands.



Diagnose

Use a review or debug skill to look for likely bugs.



Report

Evidence, changes, checks, gaps, and risk.

When Copilot fails, improve the harness.



Missing context

Add an instruction or path rule.

Wrong command

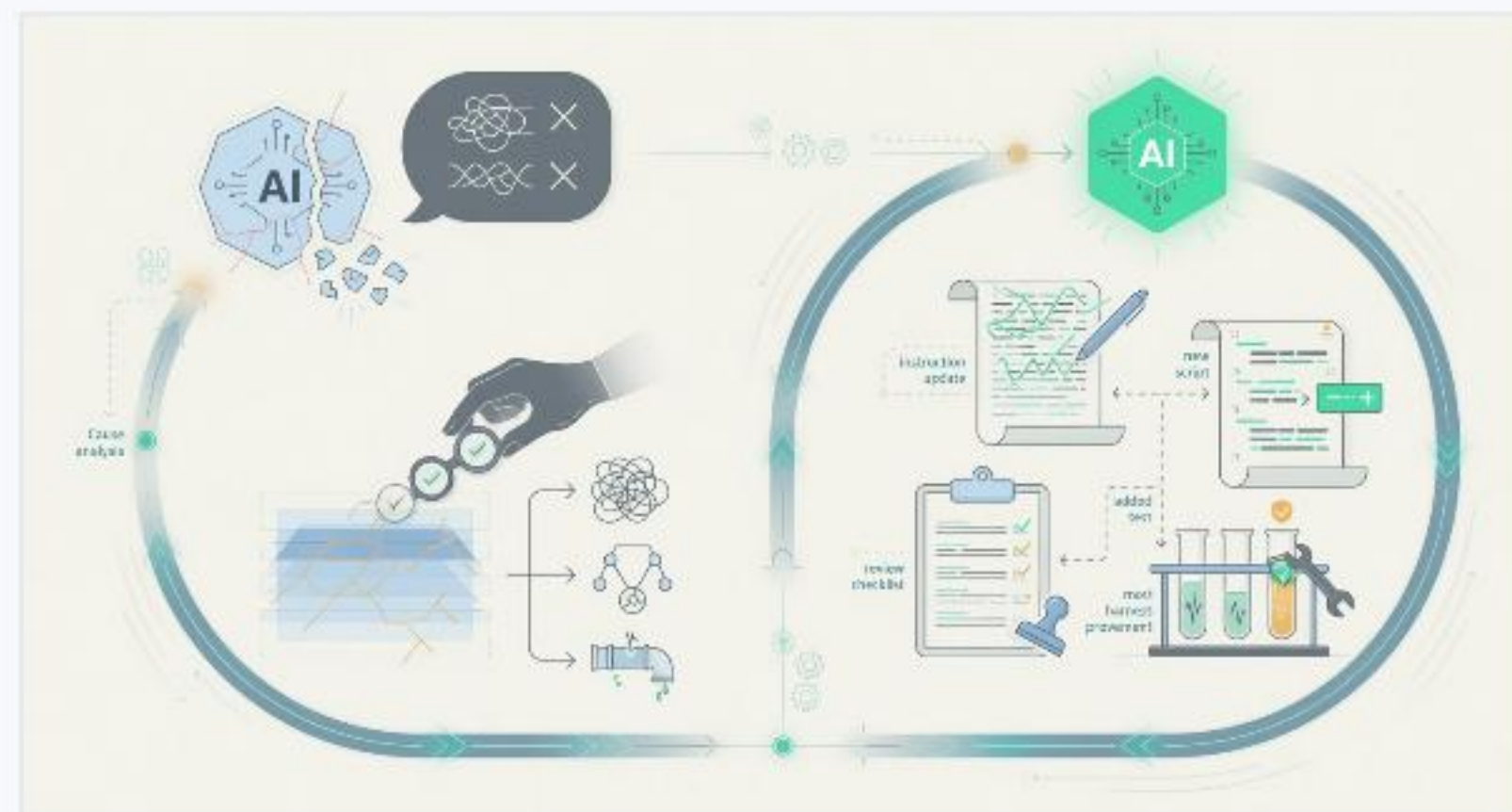
Add or document a script.

Weak proof

Add a focused test or smoke check.

Bad review

Add checklist or diagnostic skill.



Do not only fix the prompt. Fix the system that shaped the prompt.

Make AI work inspectable.

Bounded

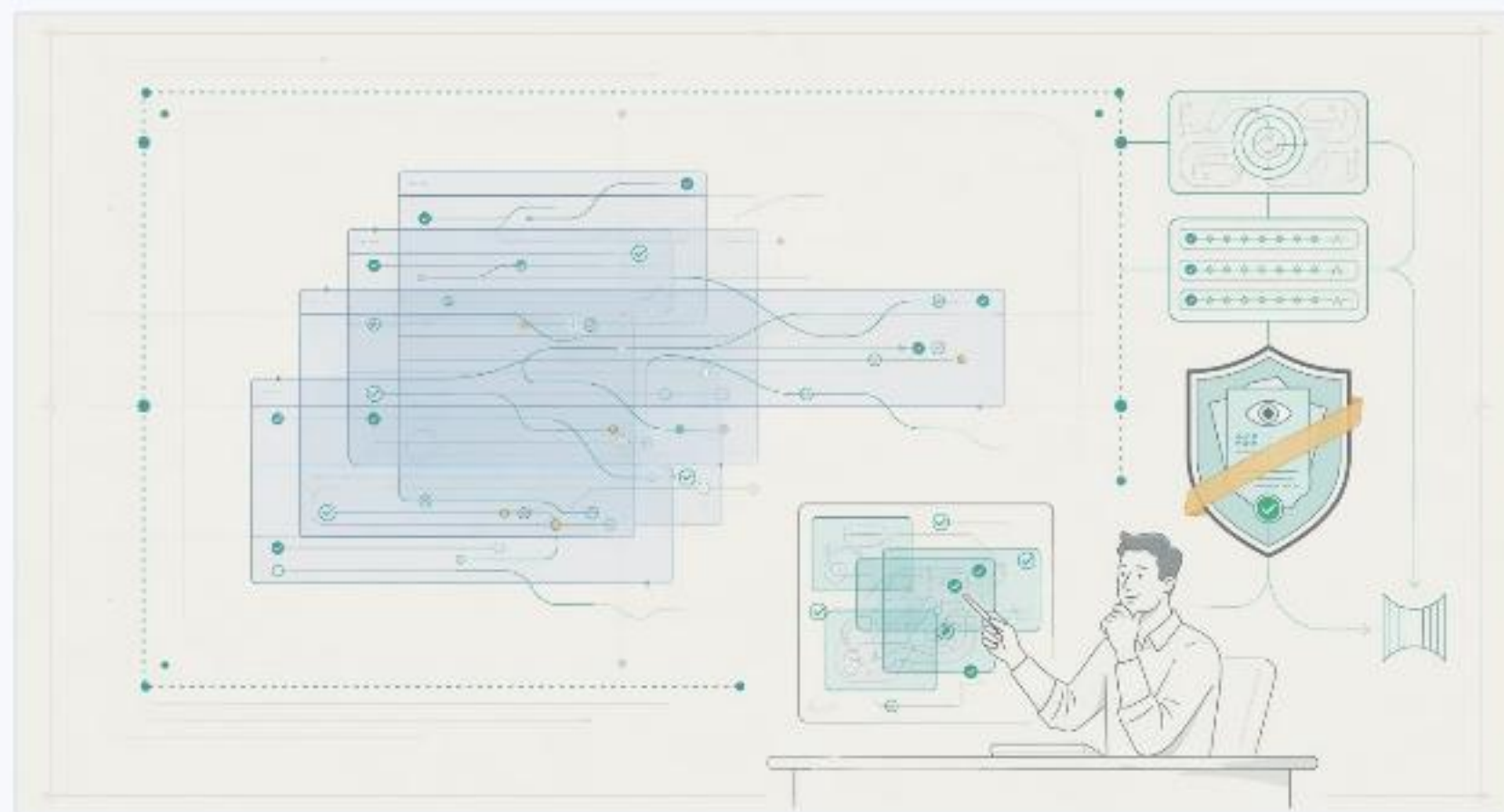
Instructions and permissions define the work area.

Visible

Skills and scripts expose the path the agent followed.

Checkable

Tests, soft reports, diagnostics, and review prove the work.



The goal is not to trust Copilot more. The goal is to make the work easier to inspect.

Fidelity Tasks VS Code demo cue

Open the Fidelity Tasks repo and show the harness parts in this order.

Rules

Copilot instructions and path-specific boundaries.

Moves

Skills/playbooks such as planning, grilling, diagnosis, and review.

Tools

Changed-test script, heavy soft-lint/report script, and smoke command.

Checks

Post-generation diagnostic pass and human review checklist.

Presenter-only slide. Not included in Ara narration.